

OXVERSE ACADEMY

AI Engineering Curriculum

AI Track (3 of 3) · Cohort-based · Project-driven

DURATION
2 Months (8 Weeks)

TOTAL WEEKS
8

LEVEL
Intermediate to Industry Ready

PROJECTS
10+

CAPSTONE
1 Production AI Application

PROGRAMME GOAL

Train engineers to design, build, evaluate, secure, and ship production-grade AI applications powered by large language models, retrieval systems, and autonomous agents.

OVERVIEW

An intensive, project-driven program covering the full AI engineering stack: Python fundamentals for AI, LLM internals, prompt and structured output engineering, retrieval-augmented generation, agentic systems with tool use, fine-tuning, multimodal AI, and production deployment with observability and security. Students ship a capstone production AI application backed by FastAPI, a vector database, and an evaluation pipeline.

No 82, Century Bus Stop, Ago Palace Way, Okota, Lagos · +234 814 846 2776

Course Outline

1. **Week 1:** Python for AI & LLM Foundations
2. **Week 2:** Structured Outputs, Function Calling & Tool Use
3. **Week 3:** Retrieval-Augmented Generation (RAG) I
4. **Week 4:** RAG II: Advanced Retrieval, Evaluation & Guardrails
5. **Week 5:** Agents & Tool-Using Systems
6. **Week 6:** Fine-Tuning, Multimodal AI & Model Customization
7. **Week 7:** Production AI Backends: Performance, Observability & Security
8. **Week 8:** Capstone: Production AI Application

Python for AI & LLM Foundations

Refresh Python for AI workloads and build a rigorous mental model of how large language models actually work under the hood.

LEARNING OBJECTIVES

- Write production-quality Python for AI applications
- Explain tokenization, embeddings, and transformer architecture in depth
- Understand context windows and their practical limits
- Call multiple LLM provider APIs correctly and idiomatically

DETAILED TOPIC BREAKDOWN

1.1 Python Refresher for AI Engineers

Virtual environments with venv and uv Type hints and Pydantic models Async/await and asyncio basics

Working with JSON and dataclasses HTTP requests with httpx Error handling and retries with tenacity

Environment variable management with python-dotenv Logging with the logging module

List/dict comprehensions for data wrangling Generators and streaming iterators Working with .env secrets safely

Package management with pip and pyproject.toml

1.2 How LLMs Actually Work

What is a large language model Tokenization (BPE, SentencePiece) Vocabulary size and token limits

Embeddings and vector representations Self-attention mechanism explained Transformer architecture (encoder/decoder)

Positional encoding Next-token prediction as the core objective Temperature, top-p, top-k sampling

Context window and its impact on cost/latency Pretraining vs fine-tuning vs RLHF

Model families: GPT, Claude, Gemini, Llama, Mistral Open-source vs closed-source models

Model cards and benchmarks (MMLU, HumanEval)

1.3 Working with LLM Provider APIs

OpenAI API (chat completions, responses API) Anthropic Claude API (messages API) Google Gemini API

Open-source models via Ollama and Hugging Face API keys and authentication Rate limits and quota management

System, user, and assistant roles Multi-turn conversation state management Comparing pricing across providers

Choosing the right model for a task Local vs hosted model tradeoffs

1.4 Prompt Engineering Fundamentals

Zero-shot vs few-shot prompting Chain-of-thought prompting System prompt design patterns Role-based prompting

Delimiters and structured prompt templates Prompt injection risks (introductory) Iterative prompt refinement workflow

Prompt versioning and testing

HANDS-ON EXERCISES

- Build a CLI chatbot using the OpenAI API with conversation memory

- Tokenize sample text with tiktoken and visualize token counts
- Compare responses from three different providers for the same prompt
- Implement exponential backoff retry logic for rate-limited API calls

ASSIGNMENTS

- Write a technical explainer document on transformer self-attention with diagrams
- Build a Python script that benchmarks latency and cost across GPT, Claude, and Gemini

PROJECTS

- Multi-provider LLM playground CLI with model switching and cost tracking

WEEKLY LEARNING OUTCOMES

- Can explain LLM internals confidently in a technical interview
- Can call and switch between major LLM provider APIs
- Understands token economics and context window tradeoffs
- Writes clean async Python for AI workloads

WEEKLY ASSESSMENT

Timed quiz on transformer internals plus a working multi-provider chatbot CLI submission

Structured Outputs, Function Calling & Tool Use

Move beyond free-text generation into reliable, machine-readable outputs and model-invoked tool calls.

LEARNING OBJECTIVES

- Force LLMs to return valid structured JSON reliably
- Design and implement function/tool calling schemas
- Validate and repair malformed LLM outputs
- Build a basic tool-using assistant

DETAILED TOPIC BREAKDOWN

2.1 Structured Output Generation

JSON mode in OpenAI and Anthropic APIs Pydantic schemas for output validation Instructor library for typed LLM outputs

JSON schema design best practices Handling malformed JSON gracefully Retry-with-repair strategies

Enum constraints and structured classification Nested object extraction from unstructured text

2.2 Function Calling & Tool Definitions

Function calling API in OpenAI Tool use in Anthropic Claude Designing clear tool descriptions and parameters

Multi-tool selection logic Parallel function calling Handling tool call errors and fallbacks

Chaining multiple tool calls in one turn Security considerations for tool execution

2.3 Building Practical Tools

Weather API tool integration Database query tool with parameterized SQL Web search tool integration

Calculator and code execution tools File read/write tools with sandboxing Designing a tool registry pattern

2.4 Output Validation & Reliability

Schema validation with Pydantic v2 Guardrails-style output checking Confidence scoring heuristics

Handling hallucinated fields Logging and replaying failed generations

HANDS-ON EXERCISES

- Extract structured invoice data from unstructured text into a Pydantic model
- Build a function-calling weather assistant with a mock API
- Implement a JSON repair loop for malformed model outputs
- Chain three tool calls together to answer a compound question

ASSIGNMENTS

- Build a structured data extraction pipeline for resumes into a typed schema
- Design and document a tool registry supporting 5 distinct tools

PROJECTS

- Tool-using research assistant that can search, calculate, and summarize with structured output

WEEKLY LEARNING OUTCOMES

- Reliably extracts typed structured data from LLMs
- Designs safe, well-documented tool schemas
- Builds assistants that can call external functions
- Handles malformed model output gracefully in production

WEEKLY ASSESSMENT

Code review of a tool-calling assistant with structured output validation

Retrieval-Augmented Generation (RAG) I

Ground LLMs in external knowledge using embeddings, chunking strategies, and vector search.

LEARNING OBJECTIVES

- Understand and implement document chunking strategies
- Generate and store embeddings in vector databases
- Build a basic RAG pipeline end-to-end
- Evaluate retrieval quality

DETAILED TOPIC BREAKDOWN

3.1 RAG Fundamentals

Why RAG: hallucination and knowledge cutoff problems RAG architecture overview (ingest, retrieve, generate)
Naive RAG vs advanced RAG pipelines When to use RAG vs fine-tuning vs long context RAG evaluation metrics overview

3.2 Document Processing & Chunking

Loading PDFs, HTML, Markdown, and DOCX Fixed-size chunking Recursive character text splitting
Semantic chunking strategies Chunk overlap tuning Metadata tagging per chunk Handling tables and images in documents
Chunking tradeoffs for retrieval quality

3.3 Embeddings

OpenAI text-embedding models Open-source embedding models (BGE, E5, sentence-transformers)
Embedding dimensionality and cost tradeoffs Batch embedding generation Cosine similarity and vector distance metrics
Embedding normalization Domain-specific embedding fine-tuning (overview)

3.4 Vector Databases

pgvector on PostgreSQL setup and indexing Pinecone setup and namespaces Qdrant setup and collections
HNSW and IVF indexing concepts Metadata filtering in vector search Hybrid search (keyword + vector)
Choosing a vector DB for a given use case Scaling vector search for millions of vectors

HANDS-ON EXERCISES

- Chunk a 100-page PDF using three different chunking strategies and compare results
- Generate embeddings for a document set and store them in pgvector
- Build a hybrid search query combining keyword and vector similarity in Qdrant
- Benchmark retrieval latency across pgvector, Pinecone, and Qdrant

ASSIGNMENTS

- Build an end-to-end ingestion pipeline: load, chunk, embed, and store documents
- Write a report comparing three vector databases on cost, latency, and features

PROJECTS

- Document Q&A system over a custom PDF knowledge base using pgvector

WEEKLY LEARNING OUTCOMES

- Designs effective chunking strategies for different document types
- Confidently sets up and queries pgvector, Pinecone, and Qdrant
- Understands embedding model tradeoffs
- Builds a working ingestion-to-retrieval pipeline

WEEKLY ASSESSMENT

Working RAG ingestion pipeline demo with retrieval quality walkthrough

RAG II: Advanced Retrieval, Evaluation & Guardrails

Push RAG systems to production quality with advanced retrieval techniques, rigorous evaluation, and safety guardrails.

LEARNING OBJECTIVES

- Implement advanced retrieval techniques to reduce hallucination
- Build automated RAG evaluation pipelines
- Apply guardrails against unsafe or off-topic outputs
- Optimize a RAG system for accuracy and latency

DETAILED TOPIC BREAKDOWN

4.1 Advanced Retrieval Techniques

Query rewriting and expansion HyDE (Hypothetical Document Embeddings) Multi-query retrieval
 Re-ranking with cross-encoders Maximal marginal relevance (MMR) Parent-document and small-to-big retrieval
 Contextual compression Self-querying retrievers with metadata filters

4.2 RAG Evaluation

Retrieval metrics: precision@k, recall@k, MRR Generation metrics: faithfulness, relevance, groundedness
 RAGAS evaluation framework Building golden datasets for evaluation LLM-as-judge evaluation patterns
 Regression testing for RAG pipelines A/B testing retrieval strategies

4.3 Guardrails & Safety

Input validation and content filtering Output moderation APIs (OpenAI moderation, Guardrails AI)
 Topical guardrails (staying on-topic) PII detection and redaction Hallucination detection heuristics
 Refusal handling and fallback responses Rate limiting abusive queries

4.4 Production RAG Patterns

Citation and source attribution in responses Streaming RAG responses to the client Caching retrieval results
 Incremental document re-indexing Multi-tenant RAG data isolation Handling conflicting or outdated sources

HANDS-ON EXERCISES

- Implement query rewriting to improve retrieval on ambiguous questions
- Build a RAGAS evaluation harness against a golden Q&A dataset
- Add PII redaction to an ingestion pipeline
- Implement citation attribution in a RAG chatbot's responses

ASSIGNMENTS

- Build a full evaluation pipeline scoring faithfulness and relevance on 50 test questions
- Write a guardrails policy document and implement it in code

PROJECTS

- Production-grade support-docs chatbot with re-ranking, citations, and guardrails

WEEKLY LEARNING OUTCOMES

- Builds RAG systems resistant to hallucination
- Runs automated, repeatable RAG evaluation pipelines
- Implements practical safety guardrails
- Ships RAG features with source citations

WEEKLY ASSESSMENT

Present a RAGAS evaluation report with before/after metrics from applied improvements

Agents & Tool-Using Systems

Build autonomous, multi-step AI agents that plan, use tools, and collaborate using LangChain, LangGraph, and MCP.

LEARNING OBJECTIVES

- Understand agent architectures and reasoning loops
- Build stateful agents with LangGraph
- Integrate the Model Context Protocol (MCP) for tool interoperability
- Design multi-agent collaboration workflows

DETAILED TOPIC BREAKDOWN

5.1 Agent Fundamentals

ReAct pattern (reason + act) Plan-and-execute agents Agent memory: short-term vs long-term
 Tool selection and routing logic Agent loops and termination conditions Failure modes: infinite loops, tool misuse
 Cost and latency implications of agentic loops

5.2 LangChain Framework

LangChain core abstractions (chains, runnables) LCEL (LangChain Expression Language) Prompt templates and output parsers
 Memory modules in LangChain Retrievers and vector store integrations Callbacks and tracing hooks
 LangChain agents and toolkits

5.3 LangGraph for Stateful Agents

Graph-based agent orchestration Nodes, edges, and state schemas Conditional branching in agent graphs
 Human-in-the-loop checkpoints Persistent state and checkpointing Cyclic graphs for iterative reasoning
 Multi-agent supervisor patterns

5.4 Model Context Protocol (MCP)

What MCP is and why it matters MCP servers and clients architecture Exposing tools via an MCP server
 Connecting Claude/other clients to MCP servers Resource and prompt primitives in MCP
 Building a custom MCP server for internal tools

5.5 Multi-Agent Systems

Supervisor-worker agent patterns Agent-to-agent communication protocols Task decomposition across agents
 Shared state and blackboard patterns Debugging multi-agent failures

HANDS-ON EXERCISES

- Build a ReAct agent that solves multi-step math and lookup tasks
- Implement a LangGraph workflow with a human-approval checkpoint
- Build a minimal MCP server exposing a file-search tool

- Design a two-agent supervisor-worker pipeline for report generation

ASSIGNMENTS

- Build a LangGraph customer support agent with escalation logic
- Write an MCP server exposing at least three tools with proper schemas

PROJECTS

- Autonomous research agent that plans, searches, retrieves, and synthesizes a report

WEEKLY LEARNING OUTCOMES

- Designs and debugs multi-step agent reasoning loops
- Builds production agents with LangGraph state management
- Exposes and consumes tools via MCP
- Orchestrates multi-agent collaborative workflows

WEEKLY ASSESSMENT

Live demo of an autonomous multi-tool agent solving an open-ended research task

Fine-Tuning, Multimodal AI & Model Customization

Customize models via fine-tuning and LoRA, and extend AI applications across vision, speech, and image generation.

LEARNING OBJECTIVES

- Fine-tune LLMs using LoRA and parameter-efficient methods
- Prepare high-quality fine-tuning datasets
- Build multimodal applications using vision, speech, and image models
- Choose between prompting, RAG, and fine-tuning appropriately

DETAILED TOPIC BREAKDOWN

6.1 Fine-Tuning Fundamentals

- When to fine-tune vs prompt vs RAG
- Full fine-tuning vs parameter-efficient fine-tuning
- LoRA and QLoRA explained
- Dataset formatting for fine-tuning (JSONL)
- Fine-tuning via OpenAI's fine-tuning API
- Fine-tuning open-source models with Hugging Face PEFT
- Evaluation before/after fine-tuning
- Overfitting and catastrophic forgetting risks

6.2 Dataset Preparation

- Collecting and cleaning training examples
- Synthetic data generation with LLMs
- Data deduplication and quality filtering
- Train/validation split strategy
- Labeling and annotation workflows
- Bias detection in training data

6.3 Vision & Multimodal Models

- Vision-language models (GPT-4o vision, Gemini vision, Claude vision)
- Image understanding and OCR use cases
- Image generation APIs (DALL-E, Imagen, Stable Diffusion)
- Prompt engineering for image generation
- Document and chart understanding with vision models
- Building a visual question-answering feature

6.4 Speech & Audio AI

- Speech-to-text with Whisper API
- Text-to-speech APIs (OpenAI TTS, ElevenLabs)
- Real-time voice pipelines
- Building a voice assistant end-to-end
- Audio streaming considerations

HANDS-ON EXERCISES

- Prepare a JSONL fine-tuning dataset from support ticket transcripts
- Fine-tune an open-source model with LoRA using Hugging Face PEFT
- Build a vision feature that extracts data from receipts using GPT-4o vision
- Build a voice memo transcription and summarization pipeline with Whisper

ASSIGNMENTS

- Fine-tune a model on a custom style/task and document before/after evaluation results
- Build a multimodal feature combining vision input and structured output

PROJECTS

- Voice-enabled multimodal assistant: speech in, vision analysis, speech out

WEEKLY LEARNING OUTCOMES

- Fine-tunes models using LoRA/QLoRA appropriately
- Prepares clean, well-structured fine-tuning datasets
- Builds vision and speech-enabled AI features
- Makes informed prompting vs RAG vs fine-tuning decisions

WEEKLY ASSESSMENT

Present a fine-tuning experiment report with quantitative before/after comparisons

Production AI Backends: Performance, Observability & Security

Engineer AI backends for real-world scale: FastAPI services, cost/latency optimization, observability, and security hardening against prompt injection.

LEARNING OBJECTIVES

- Build production FastAPI backends serving AI features
- Optimize AI applications for cost and latency
- Instrument AI pipelines with observability tooling
- Defend applications against prompt injection and abuse

DETAILED TOPIC BREAKDOWN

7.1 FastAPI for AI Backends

FastAPI project structure for AI services Async endpoints for LLM calls Server-sent events and streaming responses
WebSocket support for real-time chat Request/response schemas with Pydantic Background tasks for long-running AI jobs
API authentication and rate limiting Dependency injection for shared AI clients

7.2 Cost & Latency Optimization

Token usage tracking and budgeting Prompt compression techniques Choosing smaller models for simple tasks (model routing)
Response caching strategies (semantic caching) Batch processing for non-realtime workloads
Streaming to reduce perceived latency Parallelizing independent LLM calls Cost monitoring dashboards

7.3 Observability with LangSmith

Tracing LLM calls end-to-end Debugging chains and agent runs visually Logging prompts, completions, and metadata
Dataset creation from production traces Automated evaluation runs in LangSmith Alerting on latency and error spikes
A/B testing prompts in production

7.4 Security & Prompt Injection Defense

Direct vs indirect prompt injection Jailbreak attempts and mitigation strategies Sandboxing tool execution
Input/output sanitization Least-privilege tool permissions Data exfiltration risks via RAG documents
Red-teaming your own AI application Secrets management for API keys

7.5 Deployment & MLOps

Containerizing AI services with Docker Deploying FastAPI AI backends (Railway, Fly.io, AWS) CI/CD for AI applications
Environment-based configuration (dev/staging/prod) Model version pinning and rollback strategy Load testing AI endpoints
Monitoring uptime and error rates in production

HANDS-ON EXERCISES

- Build a streaming FastAPI chat endpoint with SSE
- Implement semantic caching to reduce duplicate LLM calls
- Instrument a LangChain pipeline with LangSmith tracing
- Attempt a prompt injection attack on your own RAG app and patch it

ASSIGNMENTS

- Deploy a FastAPI AI backend to a cloud provider with CI/CD
- Write a security assessment report covering prompt injection defenses implemented

PROJECTS

- Production-ready AI backend with streaming, caching, tracing, and deployed CI/CD pipeline

WEEKLY LEARNING OUTCOMES

- Ships FastAPI backends serving real AI features at scale
- Reduces cost and latency through concrete optimization techniques
- Uses LangSmith for full pipeline observability
- Hardens AI applications against common security threats

WEEKLY ASSESSMENT

Deployed AI backend review including load test results and security checklist

Capstone: Production AI Application

Design, build, evaluate, secure, and deploy a complete production AI application integrating everything learned in the track.

LEARNING OBJECTIVES

- Architect a complete production AI system from scratch
- Integrate RAG, agents, and structured outputs into one cohesive product
- Apply evaluation, guardrails, and observability throughout
- Deploy and present a polished, working AI application

DETAILED TOPIC BREAKDOWN

8.1 Capstone Planning & Architecture

- Choosing a capstone problem with real user value
- System architecture diagramming
- Data source identification and licensing
- Tech stack selection (FastAPI, vector DB, frontend)
- Scoping MVP vs stretch features
- Defining success metrics upfront

8.2 Core Build Sprint

- Ingestion and RAG pipeline implementation
- Agent/tool integration for multi-step tasks
- Structured output contracts for the frontend
- Streaming chat interface implementation
- Authentication and multi-user data isolation
- Persisting conversation and document history

8.3 Evaluation, Guardrails & Hardening

- Building a golden evaluation dataset for the capstone domain
- Running RAGAS or custom evaluation suite
- Adding moderation and prompt injection defenses
- Cost and latency profiling under load
- Fixing top failure cases found during testing

8.4 Deployment, Observability & Launch

- Dockerizing and deploying the full stack
- Setting up LangSmith tracing in production
- Configuring monitoring and alerting
- Writing technical documentation and architecture diagrams
- Preparing a live demo and pitch deck
- Portfolio presentation and code walkthrough

HANDS-ON EXERCISES

- Write an architecture decision record (ADR) for the capstone tech stack
- Run a full evaluation suite and document results before final polish
- Conduct a peer red-team session against your deployed capstone

ASSIGNMENTS

- Submit a complete capstone architecture document with diagrams
- Submit final evaluation report with metrics and remediation notes

PROJECTS

- Capstone: fully deployed production AI application with RAG, agents, evaluation, and observability

WEEKLY LEARNING OUTCOMES

- Ships a complete, deployed, production-grade AI application solo
- Demonstrates mastery of RAG, agents, evaluation, and security
- Presents technical work clearly to both technical and non-technical audiences
- Has a portfolio-ready capstone project for job applications

WEEKLY ASSESSMENT

Live capstone demo and defense in front of instructors and peers, graded on architecture, functionality, evaluation rigor, and presentation